

**Автономная некоммерческая профессиональная образовательная организация
«Хекслет колледж»**

**ОТЧЕТНАЯ ДОКУМЕНТАЦИЯ
по учебной практике ПМ02**

Специальность	09.02.07 «Информационные системы и программирование»
ПМ.02	Осуществление интеграции программных модулей
Студент	Драпов Матвей Борисович
Курс	3
Группа №	01-23 ИСИП ОФ 9
Период практики	04.05.2026 - 17.05.2026

Место прохождения практики: АНПОО «Хекслет колледж»

(наименование)

190020, г. Санкт-Петербург, Рижский проспект, дом № 8, литер А

(адрес)

Руководитель практики от образовательной организации

Шандыба Дарья Александровна (_____)
(ФИО) (подпись)

Результат промежуточной аттестации	
------------------------------------	--

Санкт-Петербург
2026г

**Автономная некоммерческая профессиональная образовательная организация
«Хекслет колледж»**

**ЗАДАНИЕ
на учебную практику ПМ02**

Для обучающегося	Драпова Матвея Борисовича
Специальность	09.02.07 «Информационные системы и программирование»
Курс	3
Группа	01-23 ИСИП ОФ 9
По ПМ.02	Осуществление интеграции программных модулей
Место прохождения практики	АНПОО «Хекслет колледж»
Период прохождения практик	04.05.2026 - 17.05.2026

Цель прохождения практики: разработка и представление проекта в рамках прохождения профессионального модуля ПМ.02 «Осуществление интеграции программных модулей».

Задачи практики:

- Разработать требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент
- Выполнить интеграцию модулей в программное обеспечение
- Выполнять отладку программного модуля с использованием специализированных программных средств
- Осуществить разработку тестовых наборов и тестовых сценариев для программного обеспечения.
- Произвести инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования

Руководитель практики от образовательной организации

Шандыба Дарья Александровна (_____)
(ФИО) (подпись)

Задание принято к исполнению _____ « 04 » мая 2026 г.
(подпись обучающегося)

**Автономная некоммерческая профессиональная образовательная организация
«Хекслет колледж»**

**ДНЕВНИК
прохождения учебной практики ПМ02
Общие сведения**

ФИО обучающегося	Драпов Матвей Борисович
Курс	3
Специальность	09.02.07 «Информационные системы и программирование»
Группа	01-23 ИСИП ОФ 9
Вид практики	Учебная
Место прохождения практики	АНПОО «Хекслет колледж»
Период прохождения практики	04.05.2026 - 17.05.2026

Учет выполняемой работы

№п/п	Содержание работы	Дата	Кол. часов	Подпись руководителя практики
	Разработать требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент	04.05.2026	8	
	Разработать требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент	05.05.2026	6	
	Выполнить интеграцию модулей в программное обеспечение	06.05.2026	8	
	Выполнить интеграцию модулей в программное обеспечение	07.05.2026	8	
	Выполнять отладку программного модуля с использованием специализированных программных средств	08.05.2026	6	
	Выполнять отладку программного модуля с использованием специализированных программных средств	11.05.2026	8	
	• Осуществить разработку тестовых наборов и тестовых сценариев для программного обеспечения.	12.05.2026	8	
	• Осуществить разработку тестовых наборов и тестовых сценариев для программного обеспечения.	13.05.2026	6	
	• Произвести инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования	14.05.2026	8	
	• Произвести инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования	15.05.2026	6	
		всего	72	

Дневник проверил: руководитель практики от образовательной организации

Шандыба Дарья Александровна (_____) « 17» мая 2026г.
(ФИО) (подпись)

**Автономная некоммерческая профессиональная образовательная организация
«Хекслет колледж»**

**АТТЕСТАЦИОННЫЙ ЛИСТ
по итогам прохождения производственной практики ПМ02**

На обучающегося	Драпова Матвея Борисовича
Специальность	09.02.07 «Информационные системы и программирование»
Курс	3
Группа	01-23 ИСИП ОФ 9
По ПМ.02	Осуществление интеграции программных модулей
Место прохождения практики	АНПОО «Хекслет колледж»
Период прохождения практик	04.05.2026 - 17.05.2026

Результаты освоения профессиональных компетенций

*Критерии оценки уровня освоения профессиональных компетенций:

- низкий уровень освоения – способность к выполнению работ под непосредственным руководством руководителя практики;
- средний уровень освоения – способность к выполнению работ при периодическом консультировании руководителя практики;
- высокий уровень освоения – способность к самостоятельному выполнению работ в соответствии с требованиями руководителя практики.

Общая оценка за производственную практику по ПМ.02 «Осуществление интеграции программных модулей» _____

Руководитель практики от образовательной организации

Шандыба Дарья Александровна (_____)
(ФИО) (подпись)

**Автономная некоммерческая профессиональная образовательная организация
«Хекслет колледж»**

ХАРАКТЕРИСТИКА

На обучающегося	Драпова Матвея Борисовича
Специальность	09.02.07 «Информационные системы и программирование»
Курс	3
Группа	01-23 ИСИП ОФ 9
По ПМ.02	Осуществление интеграции программных модулей
Место прохождения практики	АНПОО «Хекслет колледж»
Период прохождения практик	04.05.2026 - 17.05.2026

За время производственной практики обучающийся освоил (а):

ПК 2.1. Разрабатывать требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент.

ПК 2.2. Выполнять интеграцию модулей в программное обеспечение

ПК 2.3. Выполнять отладку программного модуля с использованием специализированных программных средств

ПК 2.4. Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения.

ПК 2.5. Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования

Работал (а) с 04 мая 2026 года по 17 мая 2026 года и показал(а) результаты по освоению следующих общих компетенций

	Общие компетенции	Результат (+, -)
ОК 01	Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам	
ОК 02	Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности	
ОК 03	Планировать и реализовывать собственное профессиональное и личностное развитие	
ОК 04	Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами	
ОК 05	Осуществлять устную и письменную коммуникацию на государственном языке с учетом особенностей социального и культурного контекста	
ОК 06	Проявлять гражданско-патриотическую позицию, демонстрировать осознанное поведение на основе общечеловеческих ценностей, применять стандарты антикоррупционного поведения..	
ОК 07	Содействовать сохранению окружающей среды, ресурсосбережению, эффективно действовать в чрезвычайных ситуациях	
ОК 08	Использовать средства физической культуры для сохранения и укрепления здоровья в процессе профессиональной деятельности и поддержания	

**Автономная некоммерческая профессиональная образовательная организация
«Хекслет колледж»**

ОТЧЕТ

о прохождении учебной практики ПМ02

Обучающийся	Драпов Матвей Борисович
Специальность	09.02.07 «Информационные системы и программирование»
ПМ 02	Осуществление интеграции программных модулей
Курс	3
Группа	01-23 ИСИП ОФ 9
Место прохождения практики	АНПОО «Хекслет колледж»
Период прохождения практик	04.05.2026 - 17.05.2026

1. Разрабатывал требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент

В ходе практики был разработан проект EXOFLORA — веб-приложение-энциклопедия экзотических растений. Архитектура приложения построена по клиент-серверной схеме: пользователь работает с React-интерфейсом, клиент отправляет HTTP-запросы к backend API, сервер обрабатывает запросы и возвращает данные о растениях в формате JSON.

Клиентская часть размещена в каталоге exoflowers-client и реализована на React + Vite. Точка входа main.jsx подключает BrowserRouter и корневой компонент App.jsx. В App.jsx описаны маршруты: главная страница, каталог, детальная страница растения, страница авторизации, административная панель и страница 404. Общая оболочка Layout.jsx отвечает за шапку, навигацию и вывод текущей страницы через Outlet.

Основные модули интерфейса разделены по назначению: pages/HomePage.jsx содержит главную страницу проекта, pages/CatalogPage.jsx реализует поиск и фильтрацию растений, pages/PlantDetailPage.jsx показывает карточку конкретного растения, pages/AuthPage.jsx содержит формы входа и регистрации, pages/AdminPage.jsx демонстрирует добавление и удаление записей. Компонент PlantCard.jsx используется для повторяющегося вывода карточек растений в каталоге.

API-слой клиента вынесен в модуль src/api/plantsApi.js. Он содержит функции fetchPlants, fetchPlant, createPlant и deletePlant. Эти функции являются публичным интерфейсом взаимодействия клиента с сервером и скрывают детали формирования URL, query-параметров, метода запроса и обработки ошибок.

Серверная часть размещена в каталоге exoflowers-server. Файл src/server.js создает Express-приложение, подключает cors и express.json, а также описывает REST-эндпоинты

/api/health, /api/plants, /api/plants/:slug, POST /api/plants, PUT /api/plants/:slug и DELETE /api/plants/:slug. Бизнес-логика вынесена в services/plantsService.js, подключение к PostgreSQL — в db.js, резервные данные — в data/plants.js.

В проекте не использовалась классическая объектно-ориентированная иерархия классов с наследованием. React-компоненты реализованы как функциональные компоненты, поэтому взаимодействие построено через композицию компонентов, props, локальное состояние useState и побочные эффекты useEffect. Публичными полями сущности растения являются id, slug, name, latinName, region, type, rarity, shortDescription, description, image, images и care. Внутренними данными компонентов являются состояния loading, error, plants, filters, form и message.

Алгоритм взаимодействия модулей можно представить так: пользователь вводит поисковый запрос или открывает страницу, React-компонент вызывает функцию из plantsApi.js, API-слой отправляет запрос на Express-сервер, серверный маршрут передает управление plantsService.js, сервис получает данные из PostgreSQL или fallback-массива, применяет фильтры, нормализует изображения и возвращает результат клиенту. После получения ответа компонент обновляет состояние и перерисовывает интерфейс.

2. Выполнял интеграцию модулей в программное обеспечение

Интеграция модулей выполнялась на уровне клиентской части, серверной части и обмена данными между ними. Frontend и backend являются отдельными npm-проектами, но работают как единая система: клиент запускается через Vite, сервер запускается через Express, а обмен данными выполняется по REST API.

На клиенте были подключены библиотеки react, react-dom и react-router-dom. React использовался для построения интерфейса на основе компонентов, react-dom — для монтирования приложения в DOM, react-router-dom — для маршрутизации между страницами без перезагрузки браузера. Vite использовался как инструмент сборки и локальный dev-сервер.

На сервере были подключены express, cors, dotenv и pg. Express использовался для создания API и маршрутов, cors — для разрешения запросов с клиентского dev-сервера, dotenv — для чтения переменных окружения, pg — для подключения к PostgreSQL при наличии DATABASE_URL. Nodemon применялся в режиме разработки для автоматического перезапуска сервера после изменений.

Взаимодействие клиента с сервером реализовано через функции fetchPlants, fetchPlant, createPlant и deletePlant. Например, каталог вызывает fetchPlants(filters), функция преобразует объект фильтров в URLSearchParams и отправляет GET-запрос на /api/plants. Сервер принимает query-параметры search, region, type и rarity, передает их в сервисный слой и возвращает массив растений.

Для устойчивой работы без обязательного подключения базы данных реализован fallback-механизм. Если DATABASE_URL не задан, plantsService.js использует массив из data/plants.js. Если переменная окружения подключена, сервис выполняет SQL-запросы через db.js. Такой подход позволил интегрировать модуль базы данных постепенно и сохранить работоспособность проекта в демонстрационном режиме.

Интеграции проверялись через запуск npm run dev в клиенте и сервере, открытие страниц приложения в браузере, проверку ответа /api/health, загрузку каталога, фильтрацию растений, переходы по маршрутам и выполнение операций добавления/удаления в административном интерфейсе. Для анализа проблем использовались консоль браузера, терминал, журналы в каталоге logs и сообщения об ошибках, возвращаемые API.

3. Выполнял отладку программного модуля с использованием специализированных программных средств

Отладка проекта выполнялась поэтапно: сначала проверялся запуск backend API, затем запуск frontend-приложения, после этого проверялось взаимодействие между клиентом и сервером. Для локального запуска использовались скрипты `npm run dev`, а также PowerShell-скрипты `start.ps1` и `stop.ps1`, которые запускают обе части проекта и сохраняют журналы работы.

Для отладки серверной части использовался `nodemon`. Он позволял автоматически перезапускать Express-сервер после изменения файлов. В терминале проверялось сообщение API started on <http://localhost:5000>, а эндпоинт `/api/health` использовался как быстрый контроль доступности API. Ошибки запуска и падения процесса дополнительно анализировались по файлам `logs/server*.log`.

Для отладки клиентской части использовался Vite dev server. Он обеспечивал быструю пересборку и обновление интерфейса при изменениях. В браузере проверялись маршруты `/`, `/catalog`, `/catalog/scadoxus-multiflorus`, `/auth` и `/admin`. Также проверялись состояния загрузки, пустого результата, ошибки запроса и корректного отображения карточек растений.

Особое внимание уделялось API-взаимодействию. Проверялось, что фильтры каталога корректно преобразуются в query-параметры, сервер возвращает JSON, а компонент `CatalogPage` обновляет список растений. В `PlantDetailPage` проверялась загрузка данных одного растения, отображение изображения, характеристик, описания и блока ухода.

В результате отладки удалось добиться стабильного запуска проекта, корректной загрузки каталога из backend API, работы поиска и фильтрации, отображения детальной страницы, обработки форм авторизации и регистрации, а также демонстрационной работы админ-панели. Из отрицательных результатов были выявлены ограничения демо-версии: операции создания, обновления и удаления при подключенной PostgreSQL-базе пока не реализованы, а без базы изменения хранятся только в памяти процесса.

Дополнительно были проверены визуальные макеты и live-скриншоты. Это позволило убедиться, что интерфейс соответствует выбранной концепции Liquid Glass, страницы сохраняют единую стилистику, а основные элементы интерфейса остаются читаемыми на разных экранах.

4. Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения

Для тестирования использовались подготовленные тестовые данные о пяти растениях: кровавая лилия, радужный эвкалипт, венерина мухоловка, трупный цветок и протей королевская. У каждой записи проверялись обязательные поля: название, латинское название, регион, тип, редкость, краткое описание, полное описание, параметры ухода и изображения.

Были составлены сценарии проверки каталога: открыть страницу /catalog, убедиться в загрузке всех растений, выполнить поиск по русскому названию, латинскому названию и региону, проверить фильтры region, type и rarity, выполнить сброс фильтров и проверить сообщение о пустой выдаче при отсутствии совпадений.

Для детальной страницы проверялся сценарий перехода к статье растения: открыть карточку из каталога, загрузить данные по slug, проверить отображение названия, латинского названия, изображения, быстрых характеристик, описания, блока ухода и похожих видов. Также проверялась обработка ситуации, когда растение не найдено.

Для формы авторизации проверялись сценарии пустого ввода, корректного ввода email и пароля, регистрации с пустыми полями, регистрации с коротким паролем, регистрации с несовпадающими паролями и успешной демонстрационной регистрации. Эти сценарии позволили проверить клиентскую валидацию и вывод сообщений пользователю.

Для административной панели проверялись сценарии загрузки текущих записей, добавления новой записи с обязательными полями, попытки отправки формы без обязательных данных и удаления записи по slug. На уровне API проверялись методы GET, POST и DELETE для /api/plants.

Средствами тестирования выступали браузер, dev server Vite, backend API Express, терминал, журналы logs, визуальные скриншоты страниц и ручная проверка REST-запросов. Формальные автоматизированные unit-тесты в проекте не добавлялись, так как в рамках практики основная проверка выполнялась вручную по пользовательским сценариям.

5. Производил инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования

При разработке проекта использовался компонентный подход React. Интерфейс был разделен на страницы, общие компоненты и API-слой. Такой подход соответствует принципу разделения ответственности: страницы отвечают за конкретные пользовательские сценарии, Layout — за общий каркас приложения, PlantCard — за переиспользуемое отображение карточки, а plantsApi.js — за обмен данными с сервером.

На серверной части применено разделение на маршруты, сервисный слой и слой доступа к данным. Файл server.js отвечает за HTTP-интерфейс, plantsService.js — за бизнес-логику фильтрации, поиска, нормализации и CRUD-операций, db.js — за подключение к PostgreSQL. Это похоже на сервисный шаблон организации backend-кода и упрощает дальнейшее расширение проекта.

В коде использовались REST-принципы: получение списка растений выполняется GET-запросом, получение одной записи — GET /api/plants/:slug, создание — POST, обновление — PUT, удаление — DELETE. Ошибки обрабатываются через HTTP-статусы 400, 404 и 500, а ответы передаются в формате JSON.

Оформление frontend-кода соответствует стандартным практикам React: компоненты именуются в PascalCase, функции и переменные — в camelCase, состояние форм хранится в useState, побочные эффекты загрузки данных выполняются через useEffect, повторяющиеся данные выводятся через map с ключами. Для предотвращения обновления размонтированных компонентов применяется флаг active в эффектах загрузки.

Оформление backend-кода соответствует практикам Node.js и Express: модули подключаются через require, экспорт функций выполняется через module.exports, SQL-запросы параметризуются, а пользовательские данные передаются через req.query, req.params и req.body. В проекте используется ESLint-конфигурация для проверки JavaScript/JSX-кода на клиентской части.

При оформлении проекта соблюдались внутренние требования учебного задания: наличие README, карты кода, скриншотов, SQL-схемы, use-case диаграммы, клиентской и серверной частей, REST API и демонстрационного интерфейса. Специальные ГОСТ для исходного кода не применялись, однако отчетная документация оформляется по требованиям инструкции: Times New Roman 12, полуторный интервал, выравнивание по ширине и отсутствие цветowych выделений в итоговом тексте.

Обучающийся

ФИО

(

)«

»

2026г

подпись